Object Oriented Programming (CS1004)

Date: February 23rd 2026

Course Instructor(s)

Ms. Sumaiyah Zahid, Ms. Sobia Iftikhar, Ms. Abeer Gauher, Mr. Ghulam Qadir, Mr. Ubaidullah, Mr. Talha Shahid, Mr. Aashir Mehboob, Mr. Saif Ur Rehman, Mr. Syed Ahmed Khan

Sessional-I Exam

Total Time (Hrs): 1
Total Marks: 30
Total Questions: 3

_____ _____ _____

Roll No Section Student Signature

Do not write below this line.

CLO # 1: Discuss knowledge of underlying concepts of object-oriented paradigm like abstraction, encapsulation, polymorphism, inheritance etc.

Question # 1: [2+2+2+2+2 = 10 marks, 15 minutes]

Explain the relevant theoretical concepts and analyze the given code. Analyze the design flaw or requirement and propose the correct OOP solution.

1. In a **BankAccount** class, the **balance** variable is currently public, allowing **main**() to execute **b.balance = -5000;**.
   a. **Why** is this direct access considered a major flaw in Object-Oriented Design?
   b. **How** do you restructure the class to prevent invalid values while still allowing the **balance** to be modified?
2. You need a variable **currentLevel** that tracks the global game progress and persists even if **GameLevel** objects are created or destroyed.
   a. **Why** does a standard variable (e.g., **int** level;) fail to solve this problem?
   b. **How** do you declare this variable so that all objects share the exact same memory location?
3. A **Car** object must strictly own 4 Wheel objects. You are choosing between **Wheel* w[4]** and **Wheel w[4]**.
   a. **Why** is the pointer approach (**Wheel***) considered less efficient for strict ownership?
   b. **How** does the object array approach (**Wheel w[4]**) strictly enforce the "**Death/Strong**" relationship (i.e., if **Car** dies, **Wheels** die)?

Trace the code execution and write the output. Assume no syntax errors.

4.
```
class System {
   static int activeUsers;
   int sessionID;
   public:
   System() {
      activeUsers++;
      sessionID = activeUsers + 100;
    }
   void status() {
     cout << sessionID << " ";
   }
   static void log() {
     cout << "<" << activeUsers << "> ";
   } };
int System::activeUsers = 0;
int main() {
  System s1; System::log();
  System s2, s3;
  s2.status(); s1.status();
  System::log(); }
```

5.
```
class Container {
   int* ptr;
   public: Container(int v) {
     ptr = new int(v);
   }
   void update(int v) {
     *ptr = v;
   } void show() {
     cout << *ptr << " ";
   } };

int main() {
   Container C1(10);
   Container C2 = C1;
   C2.update(50);
   C1.show();
   C2.show();
}
```

Part 01:

Q1. • Make balance private and provide a public Setter function with validation.

Q2. • A standard variable is re-created (and reset) for every new object instance.

OR

• Declare it as a static data member.

Q3. • Pointers scatter memory (heap fragmentation) and require manual cleanup (delete).

OR

• It stores Wheels inside the Car's contiguous memory, ensuring they are automatically destroyed when the Car is destroyed.

Part 02:

Code 01:

Output: 102 101

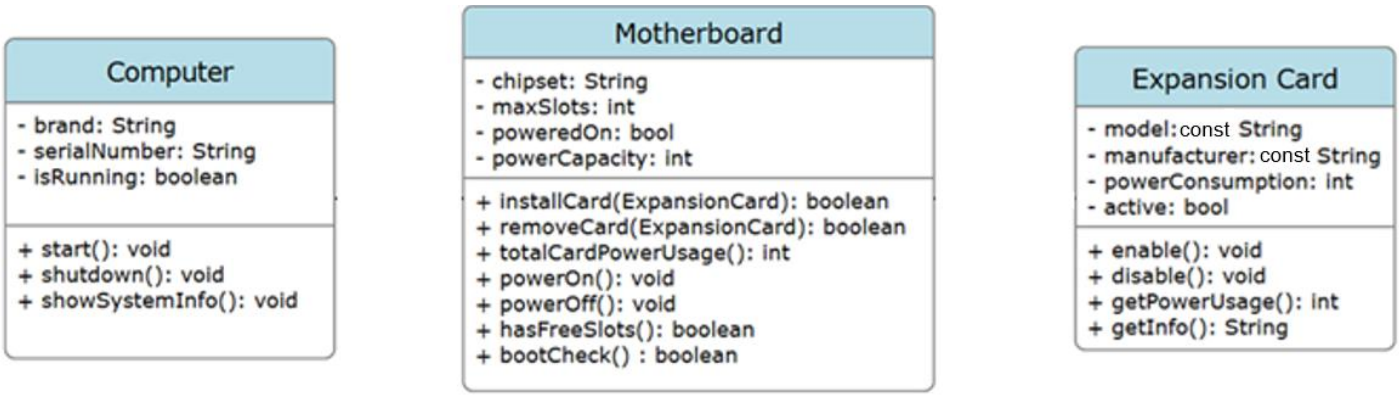
Code 02:

Logic: Default copy constructor does a Shallow Copy. C1 and C2 share the exact same pointer address. Updating C2 updates the shared memory, so C1 reflects the change too.

CLO # 4: **Illustrate Object-Oriented design artifacts and their mapping to Object-Oriented Programming using C++.**

Question # 2: **[10 marks, 25 minutes]**

You are required to design a program where a **Computer** is composed of a **Motherboard** that exists only as part of the **Computer**. The **Motherboard** can have a 3-5 **Expansion Cards** (**GPU, NIC, SoundCard, etc.**) attached to it. **Expansion Cards** are independent components that can be added to or removed from the Motherboard.



For the following tasks assume getters and setters are present for primitive datatypes and strings, unless it is stated that you are required to. Also ensure proper memory management via destructors as required.

a) **Expansion Card** Class: Perform proper encapsulation for this class, and provide one parameterized constructor which initializes all the data members. Ensure that no "default" objects or "copied" objects can be made. **(3 marks)**

b) **Motherboard** Class: You are required to implement the functionality (refer to the class diagram for the signatures): **(1+2+1+2+1 = 7 marks)**

- **Parameterized Constructor:** initializes everything except Expansion Cards.
- **hasFreeSlots():** checks if the motherboard has any free slots available. Returns true if slots are available.
- **installCard():** use the above to function to check if any slots are available. If slots are available, then installs the card in the argument to the motherboard, and returns true. If it cannot install the card, then it returns false.
- **removeCard():** removes the card received as argument from the motherboard. Returns true if the card is removed successfully. If there is no card to be removed, then it returns false.
- **bootCheck():** validates that the power consumption of all the enabled expansion cards installed in the motherboard do not exceed the power capacity of the motherboard. It should also validate that there is at least one Expansion Card. If either check fails, return false, otherwise return true.
- **powerOn():** performs a boot check. If the boot check fails, then sets the poweredOn flag to false, otherwise true.

CLO # 4: **Illustrate Object-Oriented design artifacts and their mapping to Object-Oriented Programming using C++.**

Question # 3: **[10 marks, 20 minutes]**

a) **Computer** Class: You are required to observe the signatures in the class diagram and then implement the following: **(2+2+2 = 6 marks)**

- **start():** powers on the motherboard. If the motherboard is successfully powered on, then sets the isRunning flag to true. Otherwise, sets it to false.
- **shutdown():** powers off the motherboard. Sets the isRunning flag to false.
- **showSystemInfo():** Displays the information about everything present in the system. The information about the computer, the motherboard and all the expansion cards currently present in it.

b) **Main Method: (1+1+1+1 = 4 marks)**

- Create 3 different Expansion Card Objects using DMA.
- Create 2 motherboards through DMA. Install the first Expansion Card in the first Motherboard, and the other two in the second motherboard.
- Create 2 Computers via DMA and assign one motherboard to each of them.
- Destroy the second Computer. Show that the motherboard assigned to the second computer no longer exists.

Solution:

```
#include <iostream>


using namespace std;


class ExpansionCard {
        const string model;
        const string manufacturer;
        int powerConsumption;
        bool active;
        //very important that default and copy constructors are private
        //default constructor - private to ensure that it can't be called
        ExpansionCard() : model(""), manufacturer("") {


        }
        //copy constructor - private to ensure that it can't be called
        ExpansionCard(const ExpansionCard& ec) : model(""), manufacturer("") {
        }
        public:
                ExpansionCard(string model, string manufacturer, int powerConsumption, bool active) : model(model),
manufacturer(manufacturer) {
                        this->powerConsumption = powerConsumption;
                        this->active = active;
                }
                //getters
                string getModel() {
                        return model;
                }
                string getManufacturer() {
                        return manufacturer;
                }
                int getPowerConsumption() {
                        return powerConsumption;
                }
                bool isActive(){
                        return active;
                }
                //setters
                void setPowerConsumption(int powerConsumption) {
```

```
                        this->powerConsumption = powerConsumption;
                }
                void setActive(bool active) {
                        this->active = active;
                }
};

class Motherboard {
        string chipset;
        int maxSlots;
        bool poweredOn;
        int powerCapacity;
        //aggregation:
        ExpansionCard* ecList[5];

        public:
            // Default constructor
            Motherboard() {
                chipset = "Unknown";
                maxSlots = 0;
                poweredOn = false;
                powerCapacity = 0;
                for (int i = 0; i < 5; i++) {
                        ecList[i] = nullptr;
                }
            }

            // Copy constructor
            Motherboard(const Motherboard& other) {
                chipset = other.chipset;
                maxSlots = other.maxSlots;
                poweredOn = other.poweredOn;
                powerCapacity = other.powerCapacity;
                for (int i = 0; i < 5; i++) {
                        ecList[i] = other.ecList[i];
                }
            }
```

```
        // Getters
        string getChipset() const { return chipset; }
        int getMaxSlots() const { return maxSlots; }
        bool isPoweredOn() const { return poweredOn; }
        int getPowerCapacity() const { return powerCapacity; }

        // Setters
        void setChipset(string c) { chipset = c; }
        void setMaxSlots(int slots) { maxSlots = slots; }
        void setPoweredOn(bool power) { poweredOn = power; }
        void setPowerCapacity(int capacity) { powerCapacity = capacity; }

        //functions, part b:
    // Parameterized constructor
    Motherboard(string c, int slots, bool power, int capacity) {
        chipset = c;
        maxSlots = slots;
        poweredOn = power;
        powerCapacity = capacity;
        for (int i = 0; i < 5; i++) {
                ecList[i] = nullptr;
            }
    }

    bool hasFreeSlots() {
            for (int i = 0; i < 5; i++) {
                if(ecList[i] == nullptr)
                    return true;
            }
            return false;
        }

        bool installCard(ExpansionCard* ec) {
            if(hasFreeSlots()) {
                for (int i = 0; i < 5; i++) {
                    if(ecList[i] == nullptr){
                        ecList[i] = ec;
                        return true;
```

```
                    }
                }
            } else {
                return false;
            }
        }

        bool removeCard(ExpansionCard * ec) {
            for (int i = 0; i < 5; i++) {
                if(ecList[i] == ec) {
                    ecList[i] = nullptr;
                    return true;
                }
            }
            return false;
        }

    bool bootCheck() {
            int totalpower = 0;
            for (int i = 0; i < 5; i++) {
                if(ecList[i] != nullptr) {
                    totalpower += ecList[i]->getPowerConsumption();
                }
            }
            if (totalpower < powerCapacity)
                return true;
            return false;
        }

        void powerOn() {
            if (bootCheck())
                poweredOn = true;
            poweredOn = false;
        }

        void showExpansionCardDetails() {
            for (int i = 0; i < 5; i++) {
                if (ecList[i] != nullptr)
```

```
                                cout << "Expansion Card # " << i+1 << ":\n"
                                     << "Model: " << ecList[i]->getModel() << ", "
                                     << "Manufacturer: " << ecList[i]->getManufacturer() << ", "
                                     << "Power Consumption: " << ecList[i]->getPowerConsumption() << ", "
                                     << "Active: " << ecList[i]->isActive() << endl << endl;
                            }
                        }
};

class Computer {
        string brand;
        string serialNumber;
        bool isRunning;
        Motherboard* m;
        public:
            // Default constructor
            Computer() {
                brand = "Generic";
                serialNumber = "0000";
                isRunning = false;
                m = new Motherboard();
            }

            // Parameterized constructor
            Computer(string b, string serial, bool running, Motherboard* m) {
                brand = b;
                serialNumber = serial;
                isRunning = running;
                this->m = m;
            }

            // Copy constructor
            Computer(const Computer& other) {
                brand = other.brand;
                serialNumber = other.serialNumber;
                isRunning = other.isRunning;
                m = other.m;
            }
```

```
        // Getters
        string getBrand() const { return brand; }
        string getSerialNumber() const { return serialNumber; }
        bool getIsRunning() const { return isRunning; }

        // Setters
        void setBrand(string b) { brand = b; }
        void setSerialNumber(string serial) { serialNumber = serial; }
        void setIsRunning(bool running) { isRunning = running; }

        //functions, Q4, part a:
    void start() {
            m->powerOn();
                if(m->isPoweredOn())
                    isRunning = true;
                isRunning = false;
            }
            void shutdown() {
                isRunning = false;
            }
            void showSystemInfo() {
                cout << "Computer:\nBrand: " << brand <<
                    ", Serial Number:" << serialNumber <<
                    ", isRunning: " << isRunning << endl;
                cout << "Motherboard:\nChipset: " << m->getChipset() <<
                     ", Max Slots: " << m->getMaxSlots() <<
                     ", Powered On: " << m->isPoweredOn() <<
                     ", Power Capacity: " << m->getPowerCapacity() << endl;
                m->showExpansionCardDetails();
            }
            ~Computer() {
                delete m;
            }
};

int main() {
    //Q4, part b:
```

```
        ExpansionCard *ec1 = new ExpansionCard("EC01", "MNF01", 100, false);
        ExpansionCard *ec2 = new ExpansionCard("EC02", "MNF02", 200, false);
        ExpansionCard *ec3 = new ExpansionCard("EC03", "MNF03", 150, false);

        Motherboard *mb1 = new Motherboard("CHS01", 5, false, 500);
        Motherboard *mb2 = new Motherboard("CHS02", 5, true, 200);

        mb1->installCard(ec1);
        mb2->installCard(ec2);
        mb2->installCard(ec3);

        Computer *c1 = new Computer("DELL", "SR001", false, mb1);
        Computer *c2 = new Computer("HP", "SR001", false, mb2);

        delete(c2);

        c2->showSystemInfo(); //will show garbage, might crash also
        mb2->showExpansionCardDetails(); //is a dangling pointer

        return 0;
}
```